CS5489 - Machine Learning

Lecture 4b - Supervised Learning - Regression

Dept. of Computer Science, City University of Hong Kong

Outline

1. Linear Regression
2. Selecting Features
3. **Removing Outliers**
4. Non-linear regression

In [1]:
```
# setup
%matplotlib inline
import matplotlib_inline   # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100  # display larger images
import matplotlib
from numpy import *
from sklearn import *
from scipy import stats
from mpl_toolkits.mplot3d import Axes3D
import xgboost as xgb
import torch
print(torch.__version__)
```

```
2.8.0
```

Outliers

- Too many outliers in the data can affect the squared-error term.
  - regression function will try to reduce the large prediction error for outliers, at the expense of worse prediction for other points

In [2]:
```
linbias = 30
[linX, linY, lincoefs] = datasets.make_regression(n_samples=100, n_featur
                                                  n_informative=1, bias=l
                                                  coef=True, random_state
# outliers
outbias = 10
[outX, outY, outcoefs] = datasets.make_regression(n_samples=10, n_feature
                                                  n_informative=1, bias=o
                                                  coef=True, random_state
```

```
# combine
outlinX = r_[linX, 2+outX/2]
outlinY = r_[linY, outY]

# fit using ordinary least squares
ols = linear_model.LinearRegression()
ols.fit(outlinX, outlinY)

# fit using ridge regression
alpha = 100
rr = linear_model.Ridge(alpha=alpha)
rr.fit(outlinX, outlinY)

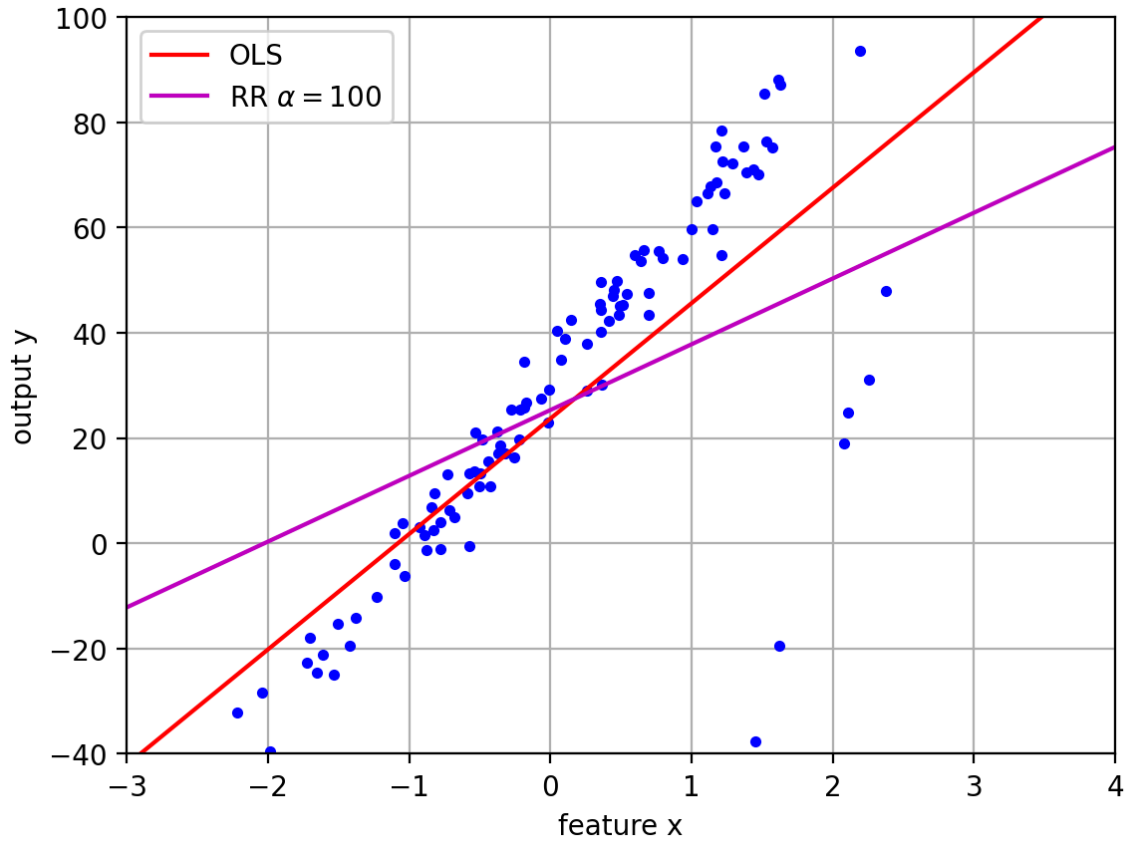
oaxbox = [-3, 4, -40, 100]

xr = linspace(oaxbox[0], oaxbox[1], 100).reshape(100,1)
ypred = ols.predict(xr)
ypredrr = rr.predict(xr)

outfig = plt.figure()
plt.plot(outlinX, outlinY, 'b.')
plt.plot(xr, ypred, 'r-', label="OLS")
plt.plot(xr, ypredrr, 'm-', label='RR $\\alpha='+str(alpha)+'$')
plt.xlabel('feature x'); plt.ylabel('output y')
plt.grid(True); plt.axis(oaxbox)
plt.legend(loc='upper left')
plt.close()
```

In [3]:
```
outfig
```

Out[3]:



RANSAC

- **RAN**dom **SA**mple **C**onsensus
  - attempt to robustly fit a regression model in the presence of corrupted data (outliers).
  - works with any regression model.
- **Idea:**
  - split the data into inliers (good data) and outliers (bad data).
  - learn the model only from the inliers

Random sampling

- Repeat many times with random subset of points (usually just enough to fit the model)
  - fit a model to the subset.
  - classify all data as inlier or outlier by calculating the residuals (prediction errors) and comparing to a threshold. The set of inliers is called the *consensus set*.
  - save the model with the highest number of inliers.
- Use the largest consensus set to learn the final model.

In [4]:
```
random.seed(1002)
ransacfig = plt.figure(figsize=(12,3))
for i in range(3):
    inds = [random.choice(range(len(outlinY))) for j in range(2)]

    ols = linear_model.LinearRegression()
    ols.fit(outlinX[inds], outlinY[inds])

    ypred = ols.predict(xr)
    T = 15

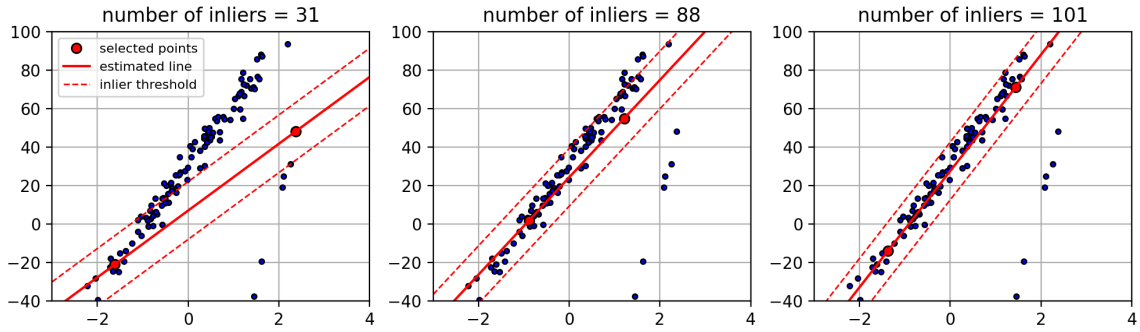
    # calculate number of inliers
    ytpred = ols.predict(outlinX)
    numin = sum(abs(ytpred-outlinY)<=T)

    plt.subplot(1,3,i+1)
    plt.plot(outlinX, outlinY, 'b.', markeredgecolor='k')
    plt.plot(outlinX[inds], outlinY[inds], 'ro', label='selected points',
    plt.plot(xr, ypred, 'r-', label='estimated line')
    plt.plot(xr, ypred+T, 'r--', linewidth=1, label='inlier threshold')
    plt.plot(xr, ypred-T, 'r--', linewidth=1)
    plt.grid(True); plt.axis(oaxbox)
    plt.title("number of inliers = " + str(numin))
    if i==0:
        plt.legend(fontsize=8, loc='upper left')


plt.close()
```

In [5]:
```
ransacfig
```

Out[5]:



RANSAC

- More iterations increases the probability of finding the correct function.
  - higher probability to select a subset of points contains all inliers.
- Threshold typically set as the median absolute deviation of y.

In [6]:
```
def plot_regr_trans_1d(model, axbox, X, Y, feattrans=None, numx=100):
    xr = linspace(axbox[0], axbox[1], numx)

    # predict the function
    if feattrans != None:
        xrf   = feattrans(xr[:,newaxis])
    else:
        xrf = xr[:,newaxis]

    Ypred = model.predict(xrf)
    hasstd = False

    if hasstd:
        plt.fill_between(xr, Ypred - 2*Ystd, Ypred + 2*Ystd,
                         alpha=0.2, color='k')

    plt.plot(X, Y, 'b.')
    plt.plot(xr, Ypred, 'r-')
    plt.axis(axbox); plt.grid(True)
```

In [7]:
```
# use RANSAC model (defaults to linear regression)
rlin = linear_model.RANSACRegressor(random_state=1234)
rlin.fit(outlinX, outlinY)

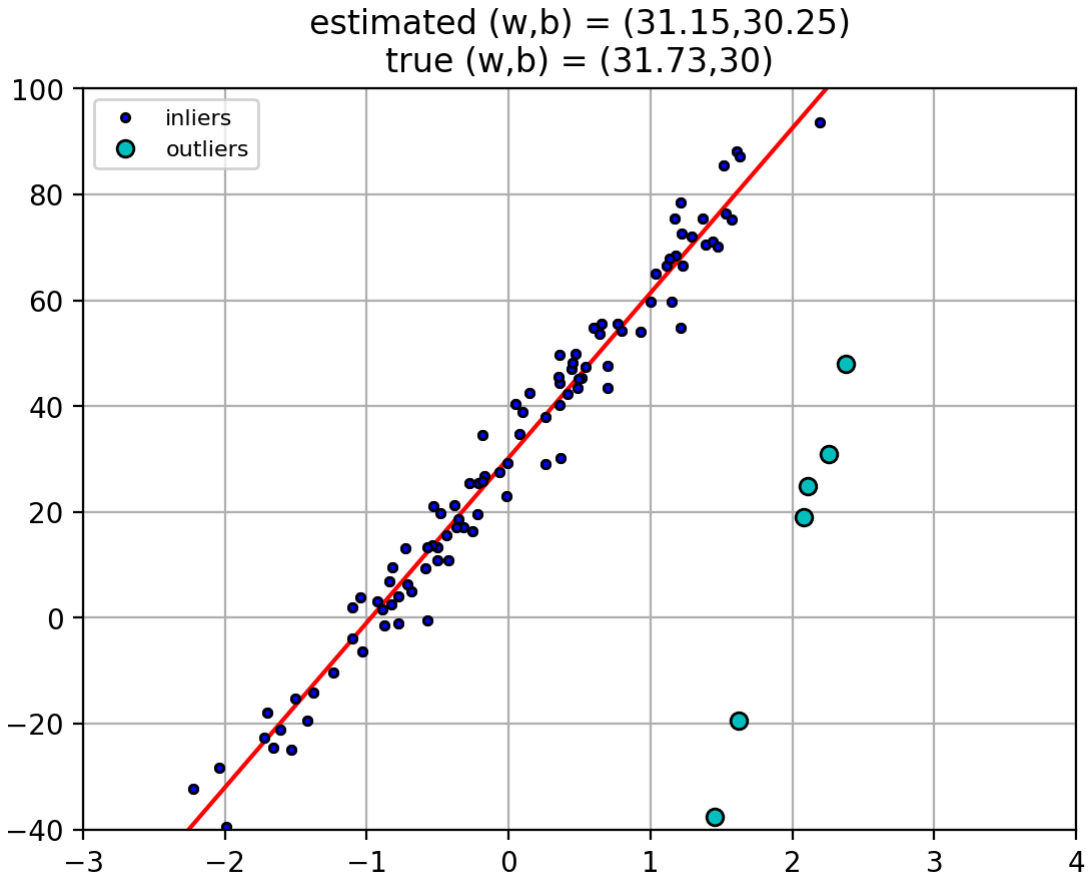
inlier_mask = rlin.inlier_mask_
outlier_mask = logical_not(inlier_mask)
```

In [8]:
```
rfig = plt.figure()
plot_regr_trans_1d(rlin, oaxbox, outlinX, outlinY)
plt.plot(outlinX[inlier_mask], outlinY[inlier_mask], 'b.', label='inliers
plt.plot(outlinX[outlier_mask], outlinY[outlier_mask], 'co', label='outli
leg = plt.legend(fontsize=8, loc='upper left')
plt.title('estimated (w,b) = (%0.4g,%0.4g)\ntrue (w,b) = (%0.4g,%0.4g)' %
          (rlin.estimator_.coef_, rlin.estimator_.intercept_, lincoefs, l
plt.close()
```

```
/var/folders/5h/9z8cfrvj02324js7fw73_cb00000gn/T/ipykernel_47931/370178953
6.py:6: DeprecationWarning: Conversion of an array with ndim > 0 to a scal
ar is deprecated, and will error in future. Ensure you extract a single el
ement from your array before performing this operation. (Deprecated NumPy
1.25.)
  plt.title('estimated (w,b) = (%0.4g,%0.4g)\ntrue (w,b) = (%0.4g,%0.4g)'
%
```

In [9]:
```
rfig
```

Out[9]:



Non-linear regression

- So far we have only considered linear regression: $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$
- Similar to classifcation, we can do non-linear regression by forming a feature vector of $\mathbf{x}$ and then performing linear regression on the feature vector.

Polynomial regression

- p-th order Polynomial function
  - $f(x) = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_p x^p$
- Collect the terms into a vector

- $f(x) = \begin{bmatrix} w_0 & w_1 & w_2 & \cdots & w_p \end{bmatrix} * \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix} = \mathbf{w}^T \phi(x)$
- weight vector $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_p \end{bmatrix}$; polynomial feature vector: $\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix}$

- Now it's a linear function, so we can use the same linear regression!

Example

- 1st to 6th order polynomials

In [10]:
```
# example data
polyX = random.normal(size=200)
polyY = sin(polyX) + 0.1*random.normal(size=200)
polyX = polyX[:,newaxis]

plin = {}
for d in [1,2,3,4,5,6]:
    # extract polynomial features with degree d
    polyfeats = preprocessing.PolynomialFeatures(degree=d)
    polyXf = polyfeats.fit_transform(polyX)

    # fit the parameters
    plin[d] = linear_model.LinearRegression()
    plin[d].fit(polyXf, polyY)
```

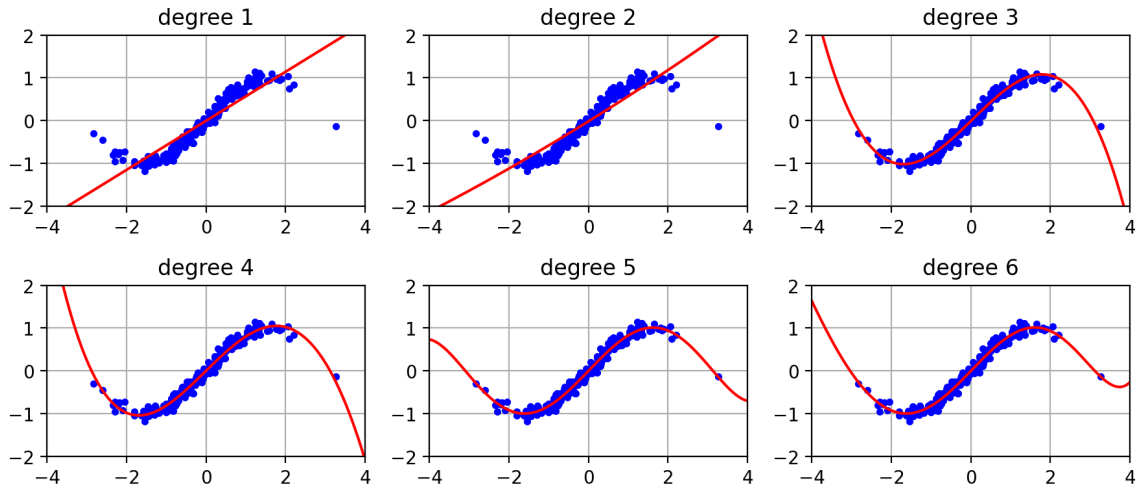
In [11]:
```
polyfig = plt.figure(figsize=(9,4))
naxbox = [-4, 4, -2, 2]

for d in [1,2,3,4,5,6]:
    # extract polynomial features with degree d
    polyfeats = preprocessing.PolynomialFeatures(degree=d)
    polyXf = polyfeats.fit_transform(polyX)

    # make plot
    plt.subplot(2,3,d)
    plot_regr_trans_1d(plin[d], naxbox, polyX, polyY, polyfeats.transform
    plt.title("degree " + str(d))
plt.tight_layout()
plt.close()
```

In [12]:
```
polyfig
```

Out[12]:



Example: Housing data

- Using AgeSold feature
- Increasing polynomial degree d will decrease MSE of training data
  - more complicated model always fits data better
  - (but it could overfit)

In [13]:

```
from sklearn.datasets import fetch_openml
def load_housing(mode=1):
    housing = fetch_openml(name="house_prices", as_frame=True)

    # 2-dim data for illustration
    if mode == 1:
        housingY = array(housing['target'])
        housingX = array(housing['data'][["GrLivArea", "YrSold", "YearBui

        # get age when sold
        housingX[:,1] -= housingX[:,2]
        housingX = housingX[:,0:2]

        ss = 4   # subsample for illustration
        housingY = housingY[::ss] / 1000
        housingX = housingX[::ss] / [[100,1]]

        housingYname = 'Home Price ($1000s)'
        housingXname = ('Living Area (100 sf)', 'Age when sold (Yrs)')
        housingXdesc = ('Living Area (100 sf)', 'Age when sold (Yrs)')
        housingaxbox = ([4.50,37,0,600], [0,140,0,600])

    elif (mode==2):
        # data for illustrating feature selection

        housingY = array(housing['target'])
        housingX = array(housing['data'][["GrLivArea", "TotRmsAbvGrd",
                                          "GarageArea", "TotalBsmtSF", "F
                                          "BedroomAbvGr", "FullBath",
                                          "LotArea", "LowQualFinSF",
                                          "OverallQual", "OverallCond",
                                          "YrSold", "YearBuilt",]])
        # get age when sold
        housingX[:,-2] -= housingX[:,-1]
```

```
        housingX = housingX[:,0:-1]

        ss = 4   # subsample for illustration
        housingY = housingY[::ss] / 1000
        housingX = housingX[::ss]

        housingYname = 'Home Price ($1000s)'
        housingXname = ["LiveA", "Rooms",
                        "GaraA", "Bsmt", "Fire",
                        "Bdrms", "Bath",
                        "LotA", "LowQA",
                        "Qual", "Cond",
                         "Age"]
        housingXdesc = ["Living Area (sqf)", "Number of Rooms", "Garage A
                       "Basement Area (sqf)", "Number of Fireplaces", "Nu
                       "Number of Bathrooms", "Lot Area (sqf)", "Area of
                       "Overall Quality of Materials", "Overall Condition
        housingaxbox = ()


    print("target: ", housingY.shape, "; data: ", housingX.shape)
    return housingX, housingY, housingXname, housingYname, housingaxbox,
```

In [14]:
```
housingX, housingY, housingXname, housingYname, housingaxbox, housingXdes
# get 2nd feature only
housingX = housingX[:,1][:,newaxis]
housingaxbox = housingaxbox[1]
housingXname = housingXname[1]
```

```
target:  (365,) ; data:  (365, 2)
```

In [15]:
```
polyfeats = {}
plin = {}
MSE = {}
for d in [1,2,3,4,5,6]:
    # extract polynomial features with degree d
    polyfeats[d] = preprocessing.PolynomialFeatures(degree=d)
    housingXf = polyfeats[d].fit_transform(housingX)

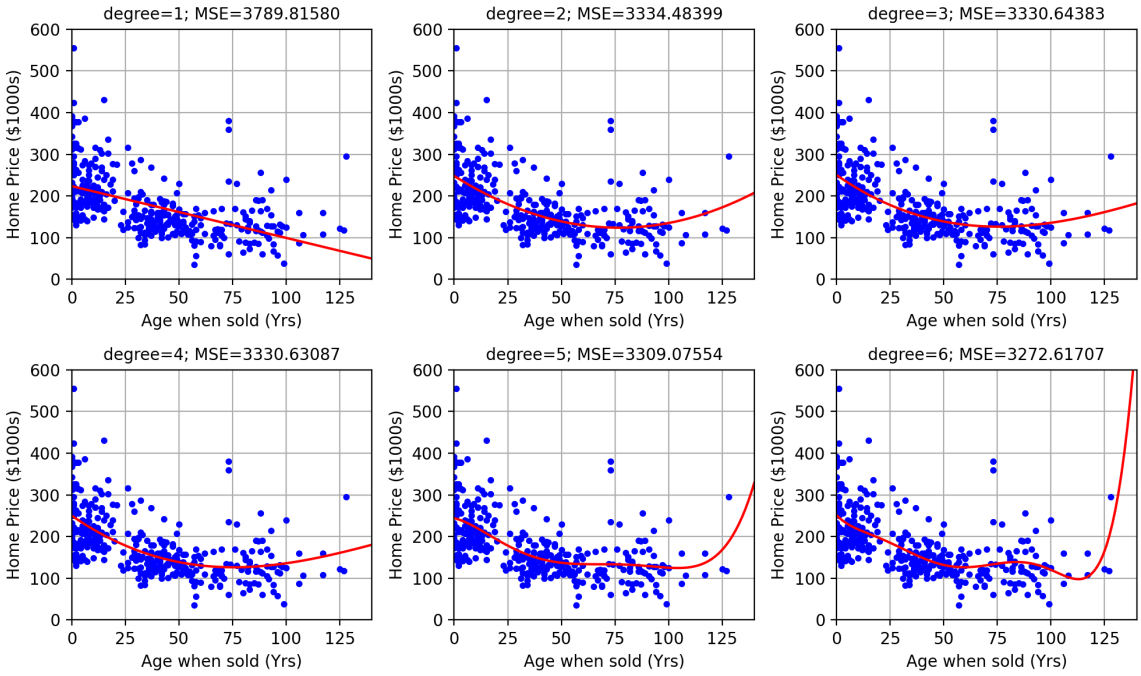
    # fit the parameters
    plin[d] = linear_model.LinearRegression()
    plin[d].fit(housingXf, housingY)

    # calculate mean-square error on training set
    MSE[d] = metrics.mean_squared_error(housingY, plin[d].predict(housing
```

In [16]:
```
pfig = plt.figure(figsize=(10,6))
for d in [1,2,3,4,5,6]:
    # make plot
    plt.subplot(2,3,d)
    plot_regr_trans_1d(plin[d], housingaxbox, housingX, housingY, polyfea
    plt.ylabel(housingYname)
    plt.title('degree={}; MSE={:.5f}'.format(d, MSE[d]), fontsize=10)
    plt.xlabel(housingXname)
plt.tight_layout()
plt.close()
```

In [17]:
```
pfig
```

Out[17]:



Select degree using Cross-Validation

- Minimizing the MSE on the training set will overfit
  - More complex function always has lower MSE on training set
- Use cross-validation to select the proper model
  - the parameters we want to change are in feature transformation step
  - Use `pipeline` to merge all steps into one object for easier cross-validation

- `pipeline` object
  - pass an array of stages
  - each entry is a tuple with the stage name and transformer (implements `.fit`, `.transform`)
  - the last entry should be a model (implements `.fit`)

In [18]:
```
# make the pipeline
polylin = pipeline.Pipeline([
        ('polyfeats', preprocessing.PolynomialFeatures(degree=1)),
        ('linreg',    linear_model.LinearRegression())
    ])
```

In [19]:
```
# set the parameters for grid search
# the parameters in each stage are named: <stage>__<parameter>
paramgrid = {
    "polyfeats__degree": array([1, 2, 3, 4, 5, 6]),
}

# do the cross-validdation search - use -MSE as the score for maximizing
plincv = model_selection.GridSearchCV(polylin, paramgrid, cv=5, n_jobs=-1
                                      scoring='neg_mean_squared_error')
plincv.fit(housingX, housingY)
```

```
print(plincv.best_params_)
```

```
{'polyfeats__degree': np.int64(2)}
```

In [20]:
```
def extract_grid_scores(modelcv, paramgrid):
    "extract CV scores from GridSearchCV and put into a matrix"
    # get parameters
    pkeys = list(paramgrid.keys())

    # initialize empty score matrix
    scoresize = [len(paramgrid[p]) for p in pkeys]
    avgscores = zeros(scoresize)

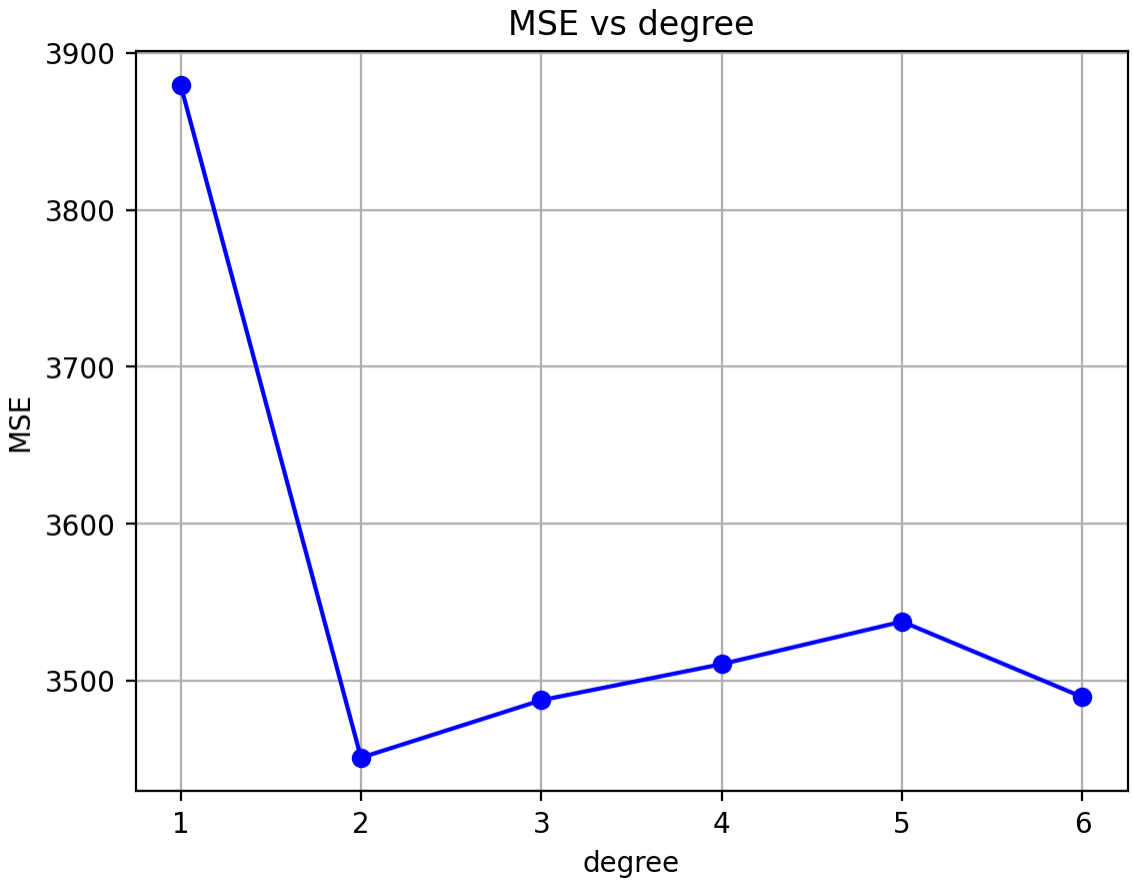
    # fill in the matrix with run for each parameter set
    for rm,rp in zip(modelcv.cv_results_['mean_test_score'], modelcv.cv_r
        # get the index into each of the parameter lists
        myind = [where(rp[p] == paramgrid[p]) for p in pkeys]
        avgscores[tuple(myind)] = rm    # put the score

    # location of best score
    bestind = [where(modelcv.best_params_[p] == paramgrid[p]) for p in pk

    return avgscores, pkeys, bestind
```

In [21]:
```
avgscores,pnames,bestind = extract_grid_scores(plincv, paramgrid)
plt.figure()
plt.plot(paramgrid['polyfeats__degree'], -avgscores, 'bo-')
plt.xlabel('degree'); plt.ylabel('MSE'); plt.grid(True)
plt.title('MSE vs degree')
```

Out[21]:
```
Text(0.5, 1.0, 'MSE vs degree')
```



Polynomial features: 2D Example

- 2D feature vectors:
  - $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}^T$
- degree 2 polynomial transformation:

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1^2 & x_1 x_2 & x_2^2 \end{bmatrix}^T$$

- degree 3 polynomial transformation:

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1^3 & x_1^2 x_2 & x_1 x_2^2 & x_3^3 \end{bmatrix}^T$$

In [22]:

```
def plot_regr_trans_2d(model, axbox, X, Y, feattrans=None):
    # maek grid
    xr = [linspace(axbox[0],axbox[1],100),
          linspace(axbox[2],axbox[3],100)]
    xgrid0, xgrid1 = meshgrid(xr[0], xr[1])
    allpts = c_[xgrid0.ravel(), xgrid1.ravel()]
    if feattrans != None:
        allptsf = feattrans(allpts)
    else:
        allptsf = allpts


    ypred = model.predict(allptsf).reshape(xgrid0.shape)
    hasstd = False


    Yr = Y.max()-Y.min()
    vmin = Y.min()-0.1*Yr
    vmax = Y.max()+0.1*Yr

    if not hasstd:
        plt.imshow(ypred, origin='lower', extent=axbox, aspect='auto',
                   vmin=vmin, vmax=vmax, cmap='jet')
        plt.scatter(X[:,0], X[:,1], c=Y, vmin=vmin, vmax=vmax, cmap='jet'

        plt.grid(True); plt.axis(axbox)
    else:
        plt.subplot(1,2,1)
        plt.imshow(ypred, origin='lower', extent=axbox, aspect='auto',
                   vmin=vmin, vmax=vmax, cmap='jet')
        plt.scatter(X[:,0], X[:,1], c=Y, vmin=vmin, vmax=vmax, cmap='jet'
        plt.grid(True); plt.axis(axbox)

        Yr = ystd.max()-ystd.min()
        vmin = ystd.min()-0.1*Yr
        vmax = ystd.max()+0.1*Yr

        plt.subplot(1,2,2)
        plt.imshow(ystd, origin='lower', extent=axbox, aspect='auto',
                   vmin=vmin, vmax=vmax, cmap='jet')
        plt.scatter(X[:,0], X[:,1], c=tmpstdX, vmin=vmin, vmax=vmax, cmap
        plt.grid(True); plt.axis(axbox)
```

In [23]:
```
housingX, housingY, housingXname, housingYname, housingaxbox, housingXdes
housingaxbox2 = housingaxbox[0][0:2]+housingaxbox[1][0:2]
```

```
target:  (365,) ; data:  (365, 2)
```

In [24]:
```
plin = {}
polyfeats = {}
MSE = {}
for i,d in enumerate([2,3]):
    # get polynomial features
    polyfeats[d] = preprocessing.PolynomialFeatures(degree=d)
    housingXf = polyfeats[d].fit_transform(housingX)

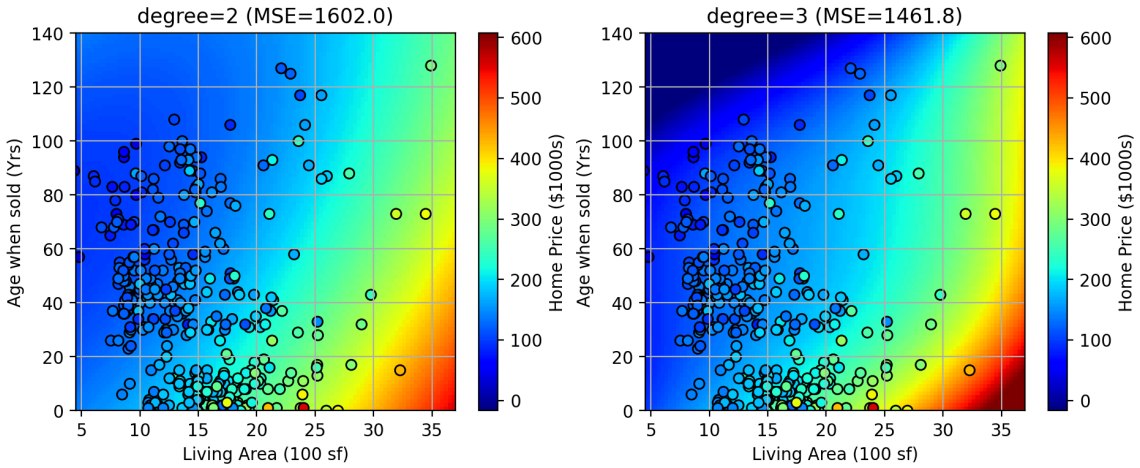
    # learn with both dimensions
    plin[d] = linear_model.LinearRegression()
    plin[d].fit(housingXf, housingY)

    # calculate MSE
    MSE[d] = metrics.mean_squared_error(housingY, plin[d].predict(housing
```

In [25]:
```
pfig = plt.figure(figsize=(11,4))
for i,d in enumerate([2,3]):
    # make figure
    plt.subplot(1,2,i+1)
    plot_regr_trans_2d(plin[d], housingaxbox2, housingX, housingY, polyfe
    cbar = plt.colorbar()
    cbar.set_label(housingYname)
    plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1])
    plt.title('degree=%d (MSE=%.1f)' % (d, MSE[d]))
plt.close()
```

In [26]:
```
pfig
```

Out[26]:



Kernel Ridge Regression

- Apply *kernel trick* to ridge regression
  - turn linear regression into non-linear regression
  - use kernel $k(x, x')$
- Closed form solution:
  - for an input point $\mathbf{x}_*$,

- prediction: $y_* = \mathbf{k}_*^T(\mathbf{K} + \alpha I)^{-1}\mathbf{y}$
  - $\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}$ is the kernel matrix ($N \times N$)
  - $\mathbf{k}_* = [k(\mathbf{x}_1, \mathbf{x}_*), \cdots, k(\mathbf{x}_N, \mathbf{x}_*)]^T$ is vector containing the kernel values between $\mathbf{x}_*$ and all training points $\mathbf{x}_i$.

Example: Polynomial Kernel

- Note: it's the same as using polynomial features and linear regression!
  - Using the kernel, we don't need to explicitly calculate the polynomial features.
  - But, we do need to calculate the kernel function between all pairs of training points.

In [27]:
```
krr = {}
for d in [1,2,3,4,5,6]:
    # fit the parameters
    krr[d] = kernel_ridge.KernelRidge(alpha=1, kernel='poly', degree=d)
    krr[d].fit(polyX, polyY)
```

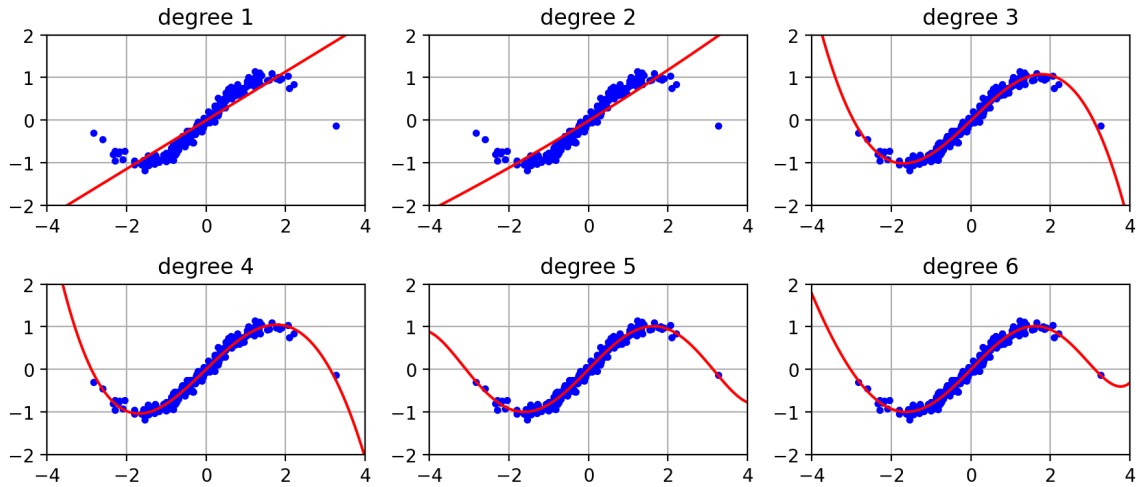
In [28]:
```
krrfig = plt.figure(figsize=(9,4))

for d in [1,2,3,4,5,6]:

    # plot the function
    plt.subplot(2,3,d)
    plot_regr_trans_1d(krr[d], naxbox, polyX, polyY)
    plt.title("degree " + str(d))
plt.tight_layout()
plt.close()
```

In [29]:
```
krrfig
```

Out[29]:



Example: RBF kernel

- gamma controls the smoothness
  - small gamma will etimate a smooth function
  - large gamma will estimate a wiggly function

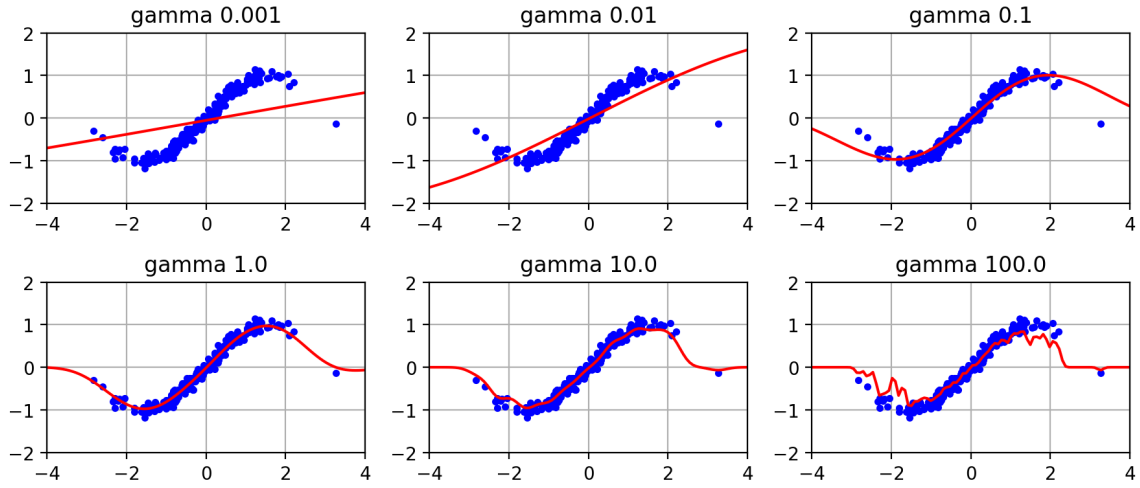
In [30]:
```
krr = {}
for i,g in enumerate(logspace(-3,2,6)):
    # fit the parameters
    krr[i] = kernel_ridge.KernelRidge(alpha=1, kernel='rbf', gamma=g)
    krr[i].fit(polyX, polyY)
```

In [31]:
```
krrfig = plt.figure(figsize=(9,4))

for i,g in enumerate(logspace(-3,2,6)):
    # plot the function
    plt.subplot(2,3,i+1)
    plot_regr_trans_1d(krr[i], naxbox, polyX, polyY)
    plt.title("gamma " + str(g))
plt.tight_layout()
plt.close()
```

In [32]:
```
krrfig
```

Out[32]:



Housing Data: Cross-validation

- RBF kernel
  - cross-validation to select α and γ.

In [33]:
```
# parameters for cross-validation
paramgrid = {'alpha': logspace(-3,3,10),
             'gamma': logspace(-3,3,10)}

# do cross-validation
krrcv = model_selection.GridSearchCV(
  kernel_ridge.KernelRidge(kernel='rbf'),  # estimator
  paramgrid,                               # parameters to try
```

```
  scoring='neg_mean_squared_error',       # score function
  cv=5,                                   # number of folds
  n_jobs=-1, verbose=True)
krrcv.fit(housingX, housingY)

print(krrcv.best_score_)
print(krrcv.best_params_)
```

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
-1616.5500709707508
{'alpha': np.float64(0.004641588833612777), 'gamma': np.float64(0.001)}
```

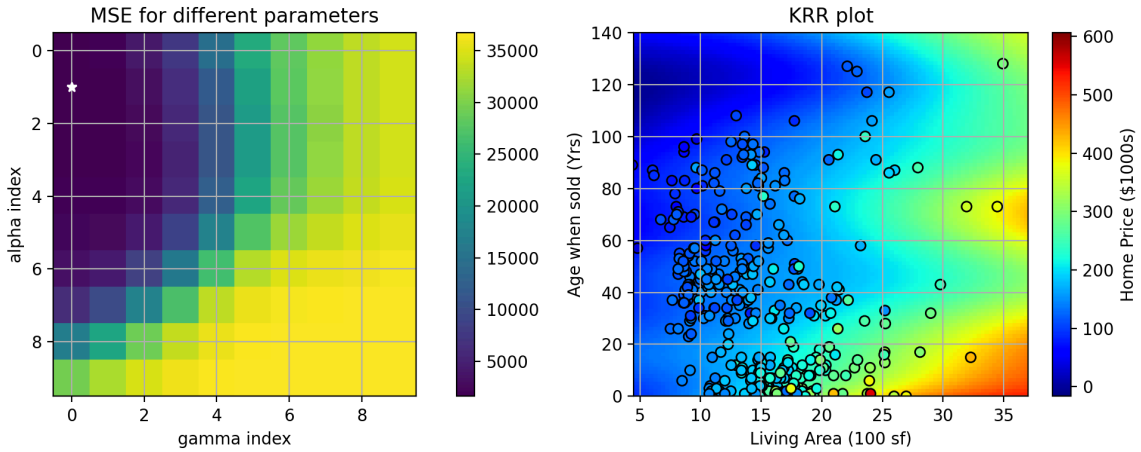
In [34]:
```
(avgscores, pnames, bestind) = extract_grid_scores(krrcv, paramgrid)

kfig = plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(-avgscores, interpolation='nearest')
plt.plot(bestind[1], bestind[0], '*w')
plt.ylabel(pnames[0] + ' index'); plt.xlabel(pnames[1] + ' index')
plt.grid(True)
plt.title('MSE for different parameters')
plt.colorbar(); plt.axis('image')

plt.subplot(1,2,2)
plot_regr_trans_2d(krrcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0])
plt.ylabel(housingXname[1])
plt.title('KRR plot')
plt.tight_layout()
plt.close()
```

In [35]:
```
kfig
```

Out[35]:



Support Vector Regression (SVR)

- Borrow ideas from classification
  - Suppose we form a "band" of width ϵ around the function:
    - if a point is inside, then it is "correctly" predicted
    - if a point is outside, then it is incorrectly predicted

In [36]:
```
def plot_svr_1d(model, axbox, X, Y, label=False, showsv=False):
    xr = linspace(axbox[0], axbox[1], 100).reshape(100,1)
    Ypred = model.predict(xr)

    epsilon = model.epsilon

    plt.plot(X, Y, 'b.', label='points')
    plt.plot(xr, Ypred, 'r-', label='$f(x)$')
    plt.plot(xr, Ypred+epsilon, 'r--', linewidth=1, label='$f(x)\\pm\\eps
    plt.plot(xr, Ypred-epsilon, 'r--', linewidth=1)

    if showsv:
        ypred = model.predict(xr)
        ss = model.support_   # get support vector indicies
        plt.plot(X[ss], Y[ss], 'r.', label='support vectors')

    plt.axis(axbox); plt.grid(True)

    if label:
        plt.legend(loc=0)
```

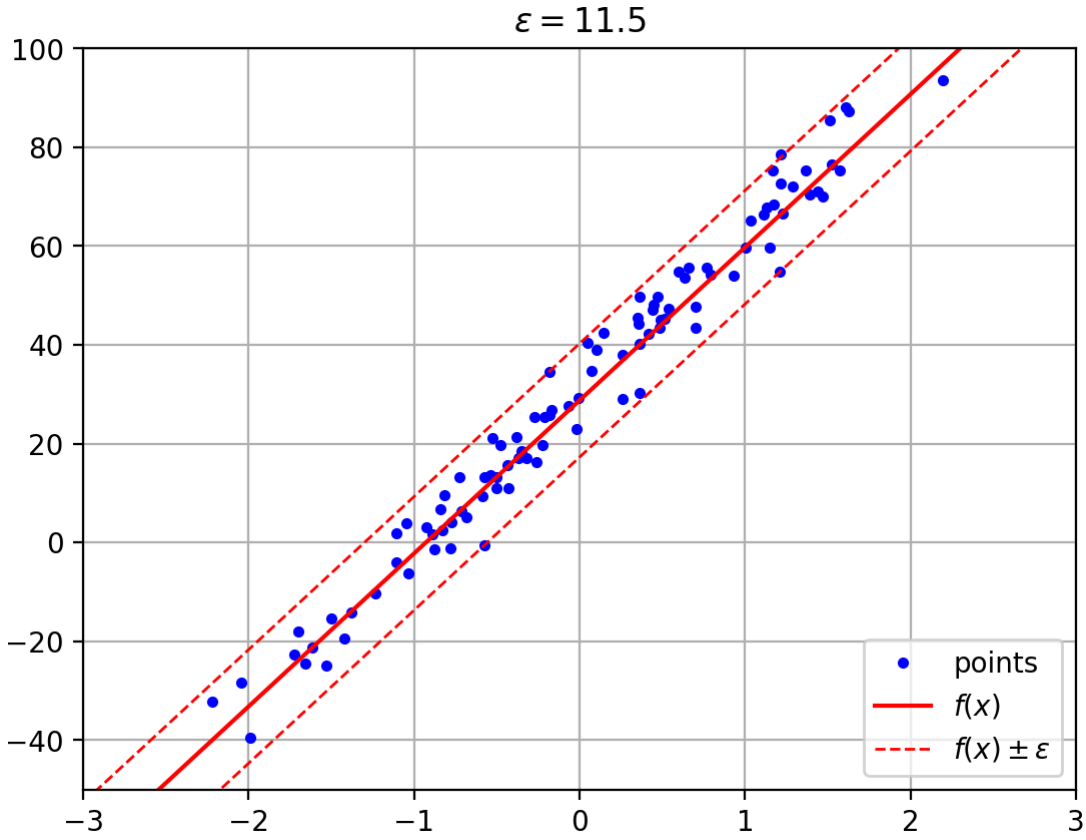
In [38]:
```
epsilon = 11.5
svr = svm.SVR(C=1000, kernel='linear', epsilon=epsilon)
svr.fit(linX, linY)

axbox = [-3, 3, -50, 100]
lsaxbox = [-3,3,-2,4]
laxbox = [-3, 3, -50, 100]

svrfig = plt.figure()
plot_svr_1d(svr, laxbox, linX, linY)
plt.grid(True); plt.axis(laxbox)
plt.legend(loc='lower right')
plt.title('$\\epsilon=11.5$')
plt.close()
```

In [39]:
```
svrfig
```

Out[39]:



- Allow some points to be outside the "tube".
  - penalty of point outside tube is controlled by C parameter.
- SVR objective function:

$$\min_{\mathbf{w},b} \sum_{i=1}^{N} \left| y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right|_\epsilon + \frac{1}{C} ||\mathbf{w}||^2$$

  - find the least complex function with most points inside the tube.
- epsilon-insensitive error:
  - $|z|_\epsilon = \begin{cases} 0, & |z| \leq \epsilon \\ |z| - \epsilon, & |z| > \epsilon \end{cases}$
- Similar to SVM classifier, the points on the band will be the *support vectors* that define the function.

Different tube widths

- The points on/outside the tube are the *support vectors*.

In [40]:

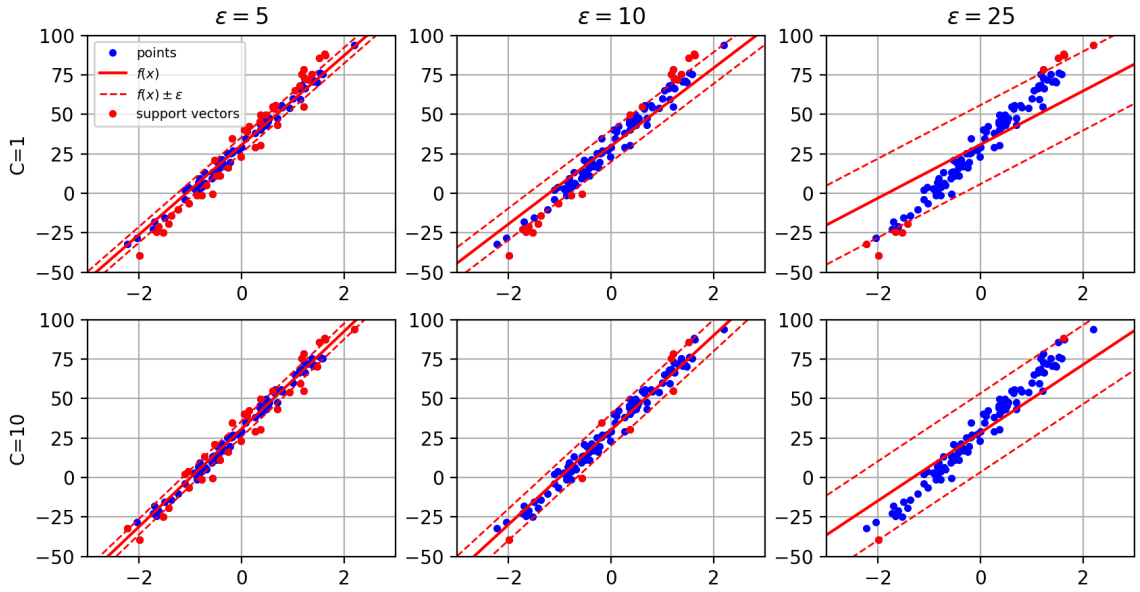
```
svrfig = plt.figure(figsize=(10,5))
ind = 1
for C in [1,10]:
    for epsilon in [5,10,25]:
        svr = svm.SVR(C=C, kernel='linear', epsilon=epsilon)
        svr.fit(linX, linY)

        plt.subplot(2,3,ind)
```

```
        plot_svr_1d(svr, axbox, linX, linY, label=False, showsv=True)
        if (C==1):
            plt.title('$\\epsilon='+str(epsilon)+"$")
        if (epsilon==5):
            plt.ylabel('C='+str(C))
        if (ind==1):
            plt.legend(fontsize=7, loc='upper left')
        ind=ind+1
plt.close()
```

In [41]:
```
svrfig
```

Out[41]:



Kernel SVR

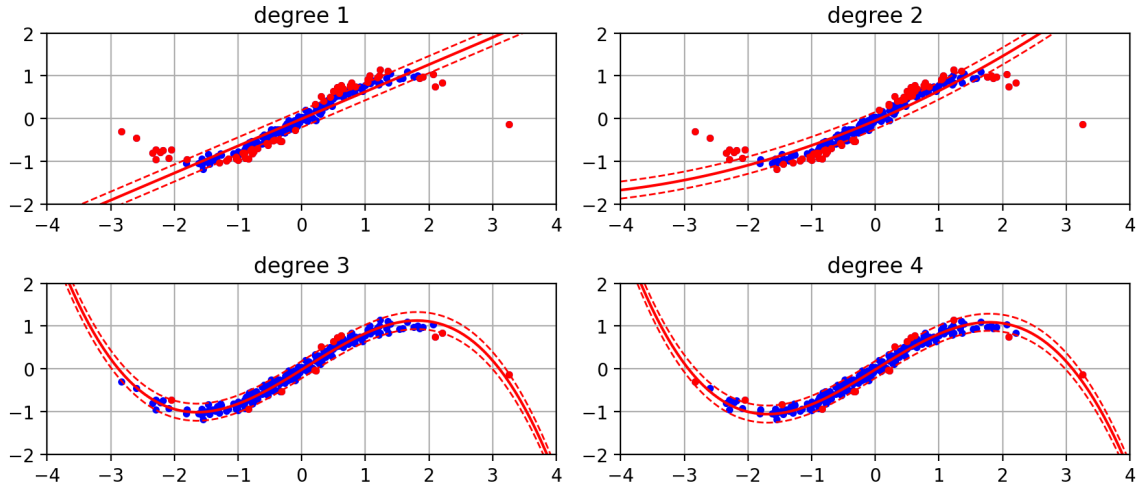
- Support vector regression can also be kernelized similar to SVM
  - turn linear regression to non-linear regression
- Polynomial Kernel:

In [42]:
```
epsilon = 0.2
svr = {}
for d in [1,2,3,4]:
    # fit the parameters (poly SVR)
    svr[d] = svm.SVR(C=1000, kernel='poly', coef0=0.1, degree=d, epsilon=
    svr[d].fit(polyX, polyY)
```

In [43]:
```
svrfig=plt.figure(figsize=(9,4))
for d in [1,2,3,4]:
    plt.subplot(2,2,d)
    plot_svr_1d(svr[d], naxbox, polyX, polyY, showsv=True)
    plt.title("degree " + str(d))
plt.tight_layout()
plt.close()
```

In [44]:
```
svrfig
```

Out[44]:



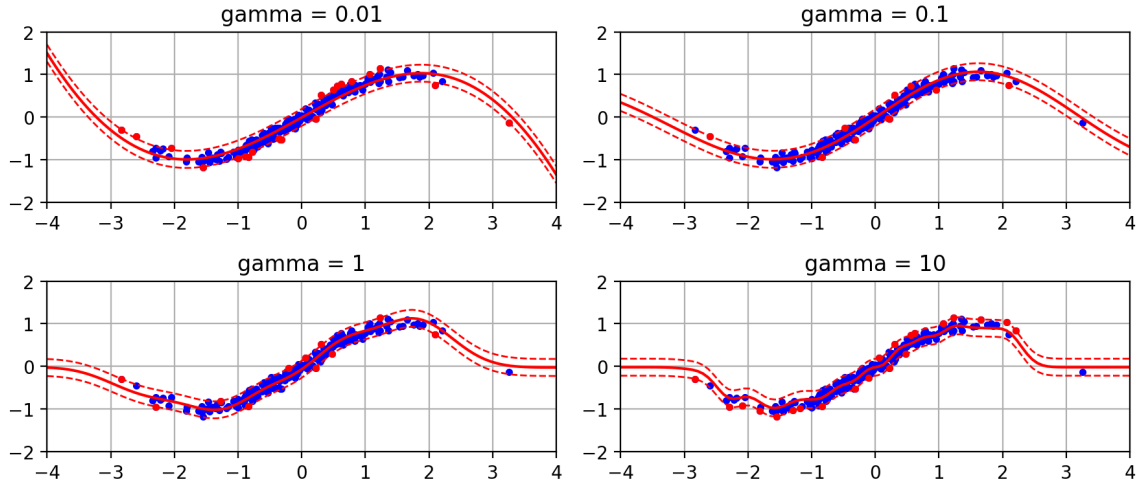
SVR with RBF kernel

In [45]:
```
epsilon = 0.2
svr = {}
for i,g in enumerate([0.01, 0.1, 1, 10]):
    # fit the parameters: SVR with RBF
    svr[i] = svm.SVR(C=1000, kernel='rbf', gamma=g, epsilon=epsilon)
    svr[i].fit(polyX, polyY)
```

In [46]:
```
svrfig = plt.figure(figsize=(9,4))
for i,g in enumerate([0.01, 0.1, 1, 10]):
    plt.subplot(2,2,i+1)
    plot_svr_1d(svr[i], naxbox, polyX, polyY, showsv=True)
    plt.title("gamma = " + str(g))
plt.tight_layout()
plt.close()
```

In [47]:
```
svrfig
```

Out[47]:



Housing Data

- Cross-validation to select 3 parameters
  - C, γ, ϵ

In [48]:
```
# parameters for cross-validation
paramgrid = {'C':       logspace(-3,3,10),
             'gamma':   logspace(-3,3,10),
             'epsilon': logspace(-2,2,10)}

# do cross-validation
svrcv = model_selection.GridSearchCV(
    svm.SVR(kernel='rbf'),  # estimator
    paramgrid,                    # parameters to try
    scoring='neg_mean_squared_error',  # score function
    cv=5,
    n_jobs=-1, verbose=1)               # show progress
svrcv.fit(housingX, housingY)

print(svrcv.best_score_)
print(svrcv.best_params_)
```

```
Fitting 5 folds for each of 1000 candidates, totalling 5000 fits
-1558.6869614323266
{'C': np.float64(215.44346900318823), 'epsilon': np.float64(12.91549665014
8826), 'gamma': np.float64(0.004641588833612777)}
```

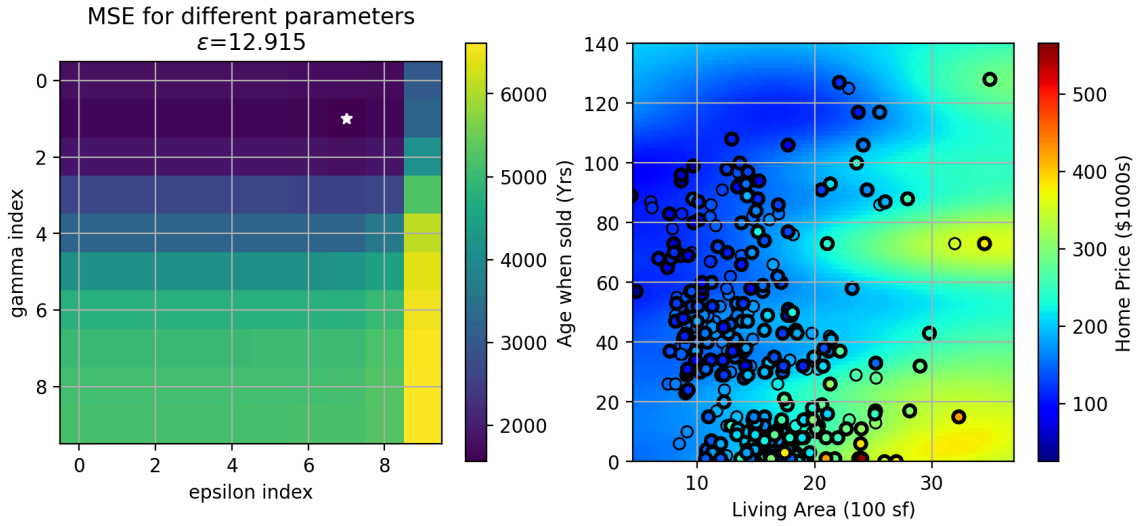
In [49]:
```
(avgscores, pnames, bestind) = extract_grid_scores(svrcv, paramgrid)

kfig = plt.figure(figsize=(10,4))
# show score map
plt.subplot(1,2,1)
plt.imshow(squeeze(-avgscores[bestind[0],:,:]), interpolation='nearest')
plt.plot(bestind[2], bestind[1], '*w')
plt.ylabel(pnames[1] + ' index'); plt.xlabel(pnames[2] + ' index')
plt.grid(True)
plt.title('MSE for different parameters\n$\epsilon$={:.3f}'.format(svrcv.
plt.colorbar(); plt.axis('image')

# show regression function
vmin = housingY.min()-10
vmax = housingY.max()+10
plt.subplot(1,2,2)
plot_regr_trans_2d(svrcv, housingaxbox2, housingX, housingY)
ss = svrcv.best_estimator_.support_
plt.scatter(housingX[ss,0], housingX[ss,1], c=housingY[ss], vmin=vmin, vm
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

In [50]:
```
kfig
```

Out[50]:



Model Ensemble

- *Why trust only one model (expert)?*
  - In real life, we may consult several experts, or go with the "wisdom of the crowd"
  - In machine learning, *why trust only one regression model?*
- Ensemble methods aim to combine multiple regression model together to form a better algorithm.
- Examples:
  - **bagging** - training multiple regression models from random selection of training data
  - **boosting** - training multiple regression models, each focusing on errors made by previous model.

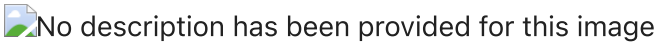
Decision Tree (Regression Tree)

- Simple "Rule-based" regression
  - At each node, move down the tree based on that node's criteria.
  - leaf node contains the prediction
- **Advantage:** can create complex conjunction of rules
- **Disadvantage:** easy to overfit by itself

Random Forest Regression

- Use **bagging** to make an ensemble of Decision Tree Regressor
- for each *Decision Tree Regressor*
  - create a new training set by randomly sampling from the training set

    - for each split in a tree, select a random subset of features to use
    - Leaf node of tree contains the predicted value.
- for a test sample, the prediction is the average value of the target variable in the leaf node that the data point lands in.



- **Advantages**
    - can do feature selection.
    - good generalization.
    - fast.
- **Disadvantages**
    - can be sensitive to outliers

In [51]:
```
def plot_rf_1d(model, axbox, X, Y, label=False, showdt=False, numx=100):
    xr = linspace(axbox[0], axbox[1], numx).reshape(numx,1)
    Ypred = model.predict(xr)

    if showdt:
        myalpha=0.25
    else:
        myalpha=1.0
    plt.plot(X, Y, 'b.', label='points', alpha=myalpha)

    if showdt:
        for i in model.estimators_:
            yy = i.predict(xr)
            plt.plot(xr, yy, 'm-')

    plt.plot(xr, Ypred, 'r-', label='forest', lw=2)
    plt.axis(axbox); plt.grid(True)

    if label:
        plt.legend(loc=0)
```

Example

- Four decision trees
    - the regressed function has "steps" because of the decision tree has a constant prediction for ranges of feature values.

In [52]:
```
rf = ensemble.RandomForestRegressor(n_estimators=4, random_state=4487, n_
rf.fit(polyX, polyY)
```

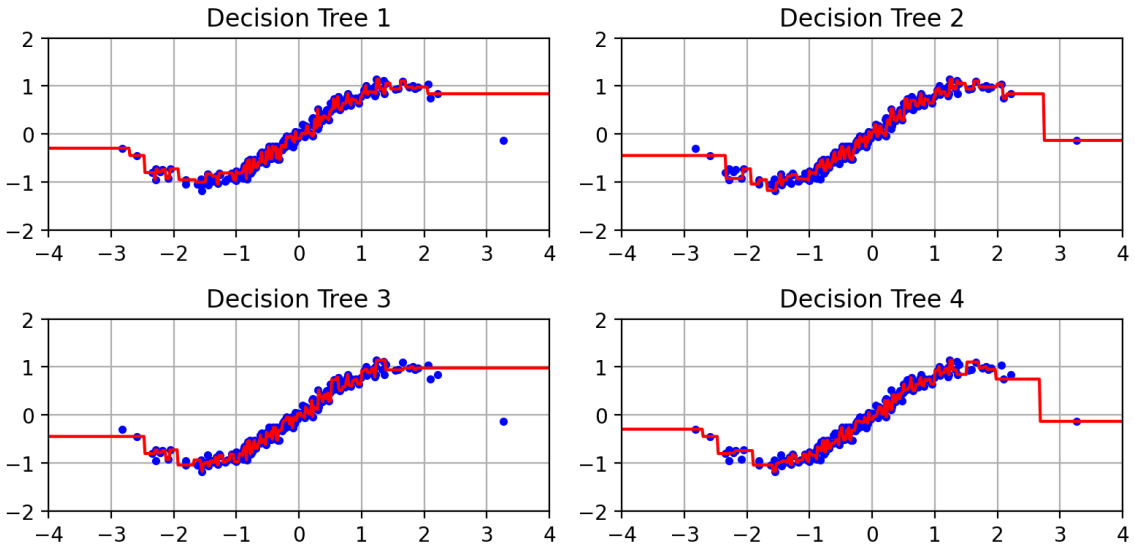
Out[52]:
```
▾ RandomForestRegressor
▶ Parameters
```

In [53]:
```
rffig = plt.figure(figsize=(8,4))
for i in range(4):
    plt.subplot(2,2,i+1)
    plot_regr_trans_1d(rf.estimators_[i], naxbox, polyX, polyY, numx=500)
```

```
    plt.title('Decision Tree ' + str(i+1))
plt.tight_layout()
plt.close()
```

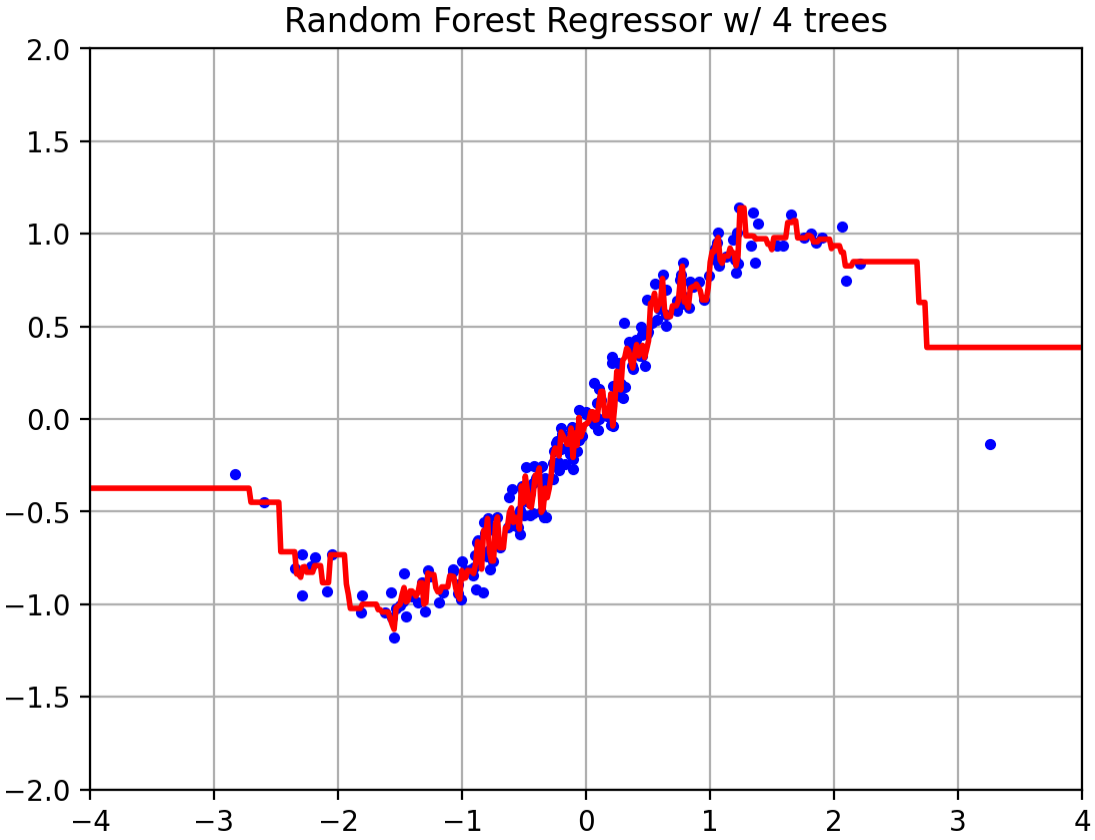
In [54]:
```
rffig
```

Out[54]:



In [55]:
```
# the aggregated function
plt.figure()
plot_rf_1d(rf, naxbox, polyX, polyY, numx=500)
plt.title('Random Forest Regressor w/ 4 trees')
```

Out[55]:
```
Text(0.5, 1.0, 'Random Forest Regressor w/ 4 trees')
```



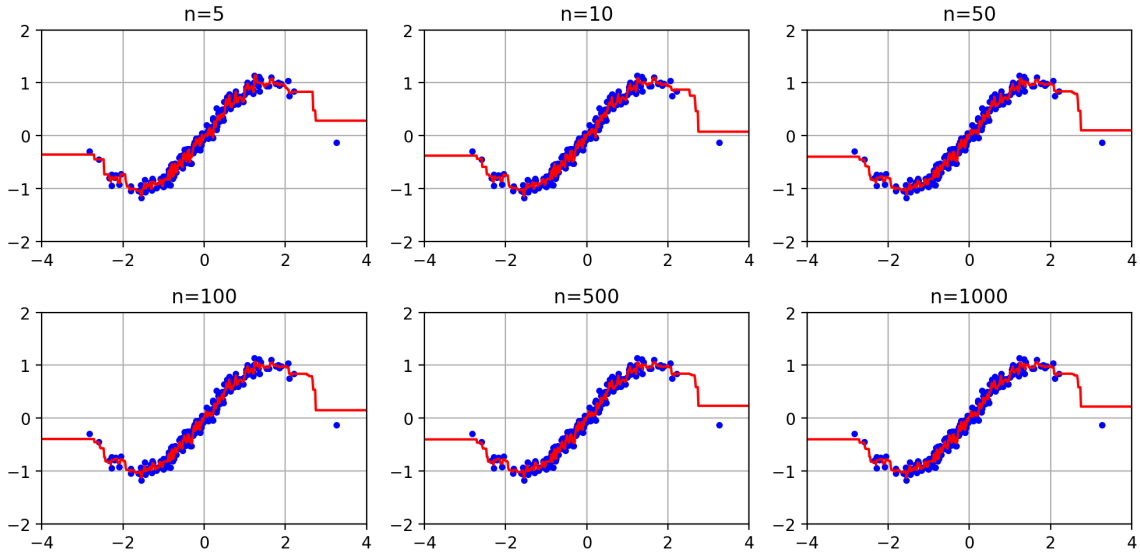
- Using more trees...

In [56]:
```
rf = {}
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):
    rf[i] = ensemble.RandomForestRegressor(n_estimators=n, random_state=4
    rf[i].fit(polyX, polyY)
```

In [57]:
```
rffig = plt.figure(figsize=(10,5))
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):
    plt.subplot(2,3,i+1)
    plot_regr_trans_1d(rf[i], naxbox, polyX, polyY, numx=500)
    plt.title('n='+str(n))
plt.tight_layout()
plt.close()
```

In [58]:
```
rffig
```

Out[58]:



Housing data

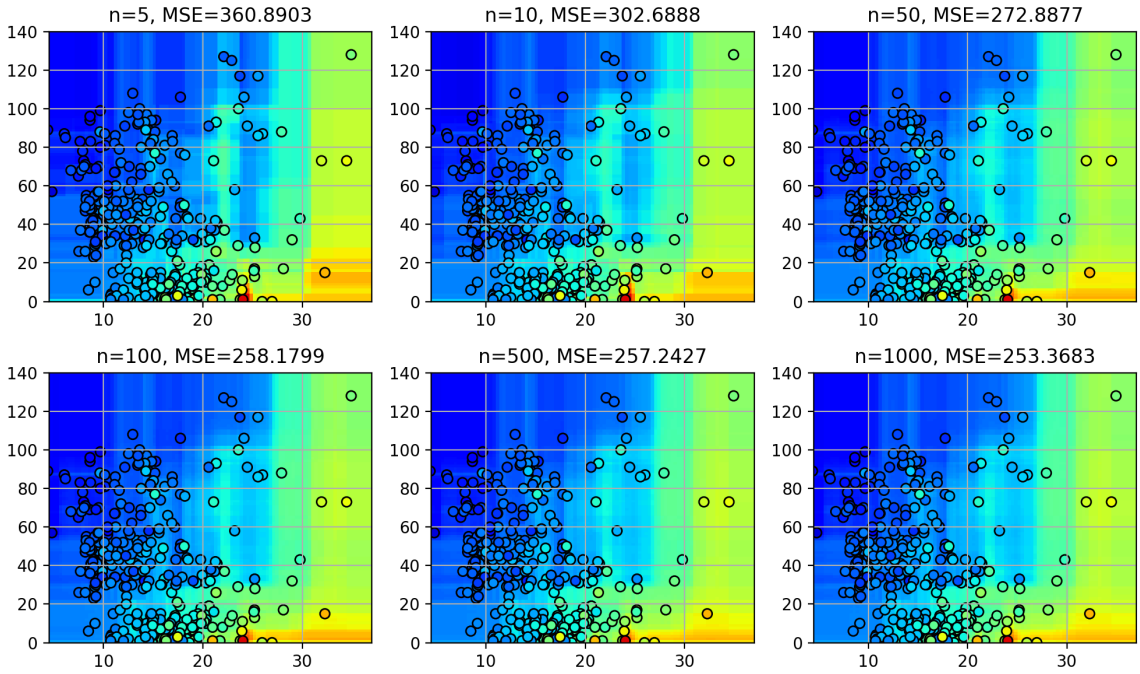
- The regressed function looks "blocky"
  - looks more reasonable for areas without any data

In [59]:
```
rf = {}; MSE = {}
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):
    rf[i] = ensemble.RandomForestRegressor(n_estimators=n, random_state=4
    rf[i].fit(housingX, housingY)
    MSE[i] = metrics.mean_squared_error(housingY, rf[i].predict(housingX)
```

In [60]:
```
rffig = plt.figure(figsize=(10,6))
for i,n in enumerate([5, 10, 50, 100, 500, 1000]):
    plt.subplot(2,3,i+1)
    plot_regr_trans_2d(rf[i], housingaxbox2, housingX, housingY)
    plt.title('n={}, MSE={:.4f}'.format(n, MSE[i]))
plt.tight_layout()
plt.close()
```

In [61]:
```
rffig
```

Out[61]:



- plot of MSE versus number of trees

In [62]:

```
ns = [1, 2, 3, 4, 5, 10, 15, 20, 25, 30, 50, 100, 500, 1000]
mses = []
for n in ns:
    rf = ensemble.RandomForestRegressor(n_estimators=n, random_state=4487
    rf.fit(housingX, housingY)

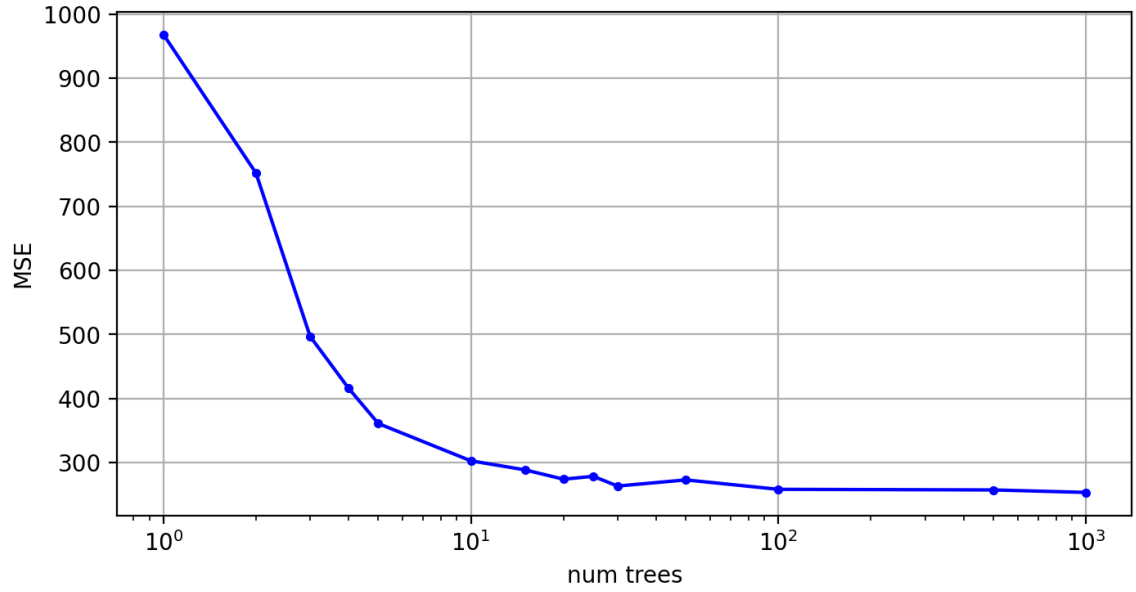
    MSE = metrics.mean_squared_error(housingY, rf.predict(housingX))
    mses.append(MSE)

mfig = plt.figure(figsize=(8,4))
plt.semilogx(ns, mses, 'b.-')
plt.xlabel('num trees')
plt.ylabel('MSE')
plt.grid(True)
plt.close()
```

In [63]:

```
mfig
```

Out[63]:



- Use cross-validation to select the tree depth

In [64]:

```
# parameters for cross-validation
paramgrid = {'max_depth': array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15]),
             }

# do cross-validation
rfcv = model_selection.GridSearchCV(
    ensemble.RandomForestRegressor(n_estimators=100, random_state=4487),
    paramgrid,                      # parameters to try
    scoring='neg_mean_squared_error',  # score function
    cv=5,
    n_jobs=-1, verbose=True
)
rfcv.fit(housingX, housingY)

print(rfcv.best_score_)
print(rfcv.best_params_)
```

```
Fitting 5 folds for each of 11 candidates, totalling 55 fits
-1717.1838417959073
{'max_depth': np.int64(5)}
```

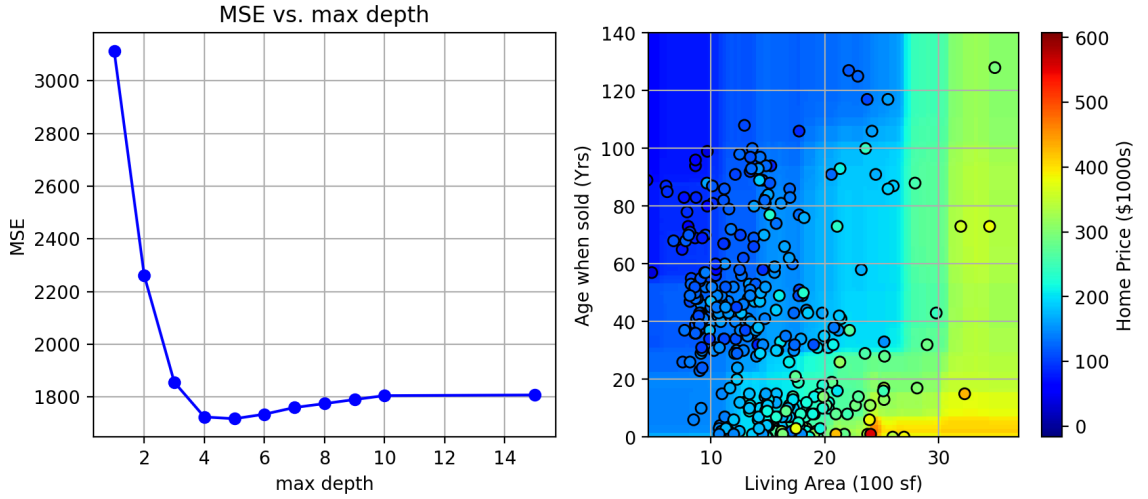
In [65]:
```
(avgscores, pnames, bestind) = extract_grid_scores(rfcv, paramgrid)

rffig = plt.figure(figsize=(10,4))
# show scores
plt.subplot(1,2,1)
plt.plot(paramgrid['max_depth'], -avgscores, 'bo-')
plt.xlabel('max depth'); plt.ylabel('MSE')
plt.title('MSE vs. max depth')
plt.grid(True)

# show regression function
plt.subplot(1,2,2)
plot_regr_trans_2d(rfcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

In [66]:
```
rffig
```

Out[66]:



Boosting

- Boosting is an ensemble learning method that combines a set of less accurate models (called "weak learners") to create a single, highly accurate model (a "strong learner")
- Boosting algorithms build models sequentially where each new model in the sequence is trained to correct the errors made by its predecessors (higher weights on samples with larger losses by predecessors).

XGBoost Regression

- variant of boosting
  - each iteration fits the residual (gap) between the current predictions and the groundtruth
  - the residual is computed as the gradient of the loss function.
- It's a gradient descent algorithm

    - in each iteration, the weak learner fits the gradient of the loss
        - $h_t(\mathbf{x}) \approx \frac{dL}{d\mathbf{f}}$
- and adds it to the function:
    - $f_t(\mathbf{x}) = f_{t-1}(\mathbf{x}) - \alpha_t h_t(\mathbf{x}) \approx f_{t-1}(\mathbf{x}) - \alpha_t \frac{dL}{d\mathbf{f}_{t-1}}$
- Generalizes boosting to other loss functions.
- Regression loss:
    - `reg:squarederror` should be used when miminizing squared loss (Gaussian noise assumption).
    - `reg:gamma` should be used when regression with Gamma noise assumption (non-negative values).
    - others...see documentation.

In [67]:
```
# setup dictionary of distributions for each parameter
paramsampler = {
    "colsample_bytree": stats.uniform(0.7, 0.3),  # default=1
    "gamma":            stats.uniform(0, 0.5),    # default=0
    "max_depth":        stats.randint(2, 6),      # default=6
    "subsample":        stats.uniform(0.6, 0.4),  # default=1
    "learning_rate":    stats.uniform(.001,1),    # default=1 (could also
    "n_estimators":     stats.randint(10, 1000),
}

# the XGB regressor using squared error loss
xr = xgb.XGBRegressor(objective="reg:squarederror", random_state=4487)

# cross-validation via random search
# n_iter = number of parameter combinations to try
xgbrcv = model_selection.RandomizedSearchCV(xr, param_distributions=param
                            scoring='neg_mean_squared_error',  # score fu
                            random_state=4487, n_iter=200, cv=5,
                            verbose=1, n_jobs=6)

xgbrcv.fit(housingX, housingY)
print("best params:", xgbrcv.best_params_)
```

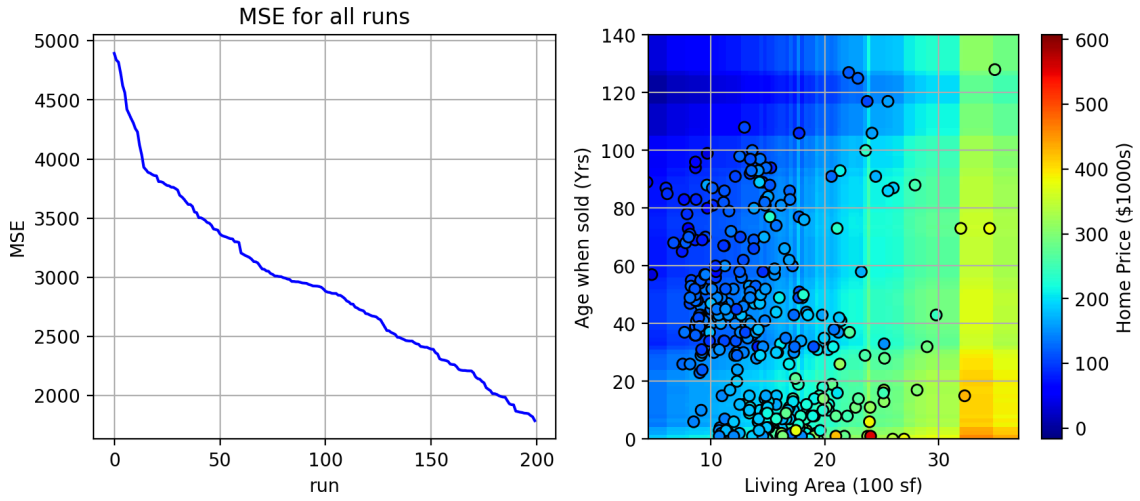
```
Fitting 5 folds for each of 200 candidates, totalling 1000 fits
best params: {'colsample_bytree': np.float64(0.8168908224647844), 'gamma':
np.float64(0.35398611989014306), 'learning_rate': np.float64(0.06248414027
218807), 'max_depth': 2, 'n_estimators': 274, 'subsample': np.float64(0.62
40108914892146)}
```

In [68]:
```
xfig = plt.figure(figsize=(10,4))
# show scores
plt.subplot(1,2,1)
plt.plot(range(200), -sort(xgbrcv.cv_results_['mean_test_score']), 'b-')
plt.xlabel('run'); plt.ylabel('MSE')
plt.title('MSE for all runs')
plt.grid(True)

# show regression function
plt.subplot(1,2,2)
plot_regr_trans_2d(xgbrcv, housingaxbox2, housingX, housingY)
cbar = plt.colorbar()
cbar.set_label(housingYname)
plt.xlabel(housingXname[0]); plt.ylabel(housingXname[1]);
plt.close()
```

In [69]:
```
xfig
```

Out[69]:



Regression Summary

- **Goal:** predict output $y \in \mathbb{R}$ from input $\mathbf{x} \in \mathbb{R}^d$.
  - i.e., learn the function $y = f(\mathbf{x})$.

Name	Function	Training	Advantages	Disadvantages
Ordinary Least Squares	linear	minimize square error between observation and predicted output.	- closed-form solution.	- sensitive to outliers and overfitting.
ridge regression	linear	minimize squared error with $\|\|w\|\|^2$ (L2-norm) regularization term.	- closed-form solution; - shrinkage to prevent overfitting.	- sensitive to outliers.
LASSO	linear	minimize squared error with $\sum_{j=1}^{d} \|w_j\|$ (L1-norm) regularization term.	- feature selection (by forcing weights to 0)	- sensitive to outliers.
OMP	linear	minimize squared error with constraint on number of non-	- feature selection	- difficult optimization problem, sensitive to outliers.

		zero weights (L0-norm).		
RANSAC	same as the base model	randomly sample subset of training data and fit model; keep model with most inliers.	- ignores outliers.	- requires enough iterations to find good consensus set.
kernel ridge regression	non-linear (kernel function)	apply "kernel trick" to ridge regression.	- non-linear regression. - Closed-form solution.	- requires calculating kernel matrix $O(N^2)$. - cross-validation to select hyperparameters.
kernel support vector regression	non-linear (kernel function)	minimize squared error, insensitive to epsilon-error.	- non-linear regression. - faster predictions than kernel ridge regression.	- requires calculating kernel matrix $O(N^2)$. - iterative solution (slow). - cross-validation to select hyperparameters.
random forest regression	non-linear (ensemble)	aggregate predictions from decision trees.	- non-linear regression. - fast predictions.	- predicts step-wise function. - cannot learn a completely smooth function.
XGBoost regression	non-linear (ensemble)	aggregaate predictions from decision trees.	- non-linear regression. - fast predictions.	- predicts step-wise function. - cannot learn a completely smooth function.

Other Things

- *Feature normalization*
  - feature normalization is typically required for regression methods with regularization.
  - makes ordering of weights more interpretable (LASSO, RR).

- *Output transformations*
  - sometimes the output values y have a large dynamic range (e.g., 10^{-1} to 10^5).
    - large output values will have large error, which will dominate the training error.
    - in this case, it is better to transform the output values using the logarithm function.
      - $\hat{y} = \log_{10}(y)$, for example, see the tutorial.

In [ ]: